INTERNSHIP PRESENTATION | PHASE-1

Smart City IoT Fog-IDS Triage Module
Edge AI & Cyber Defense Pod

Raghunatha Reddy G R | SRN: R23EA094 | B.Tech CSE (Cyber Security / AI), 6th Sem

Faculty & Workplace Guide: Dr. Supreeth S, Associate Professor, School of C&SE

Host Center: The RIO Center, REVA University | Academic Year 2025–2026

Organisation and Team

The RIO Center

• About RIO: The Research and Innovation Opportunities (RIO) Center at REVA University fosters industry-aligned incubation, advanced computing, and smart infrastructure.
• Focus Areas: Edge Artificial Intelligence, Applied Cyber Security, Embedded Systems, and Smart Urban Utilities.
• Mission: Building robust, production-grade cyber-defense solutions for smart municipal grids and campus IoT testbeds.

My team

• Pod Composition: Edge AI & Cyber Defense Pod comprising 4 student engineers covering threat modeling, network ingestion, and ML pipeline development.
• Reporting: Report directly to Faculty & Project Guide Dr. Supreeth S.
• Personal Scope: End-to-end responsibility for the fog-tier traffic ingestion pipeline, feature normalization, and sub-millisecond Random Forest triage module.

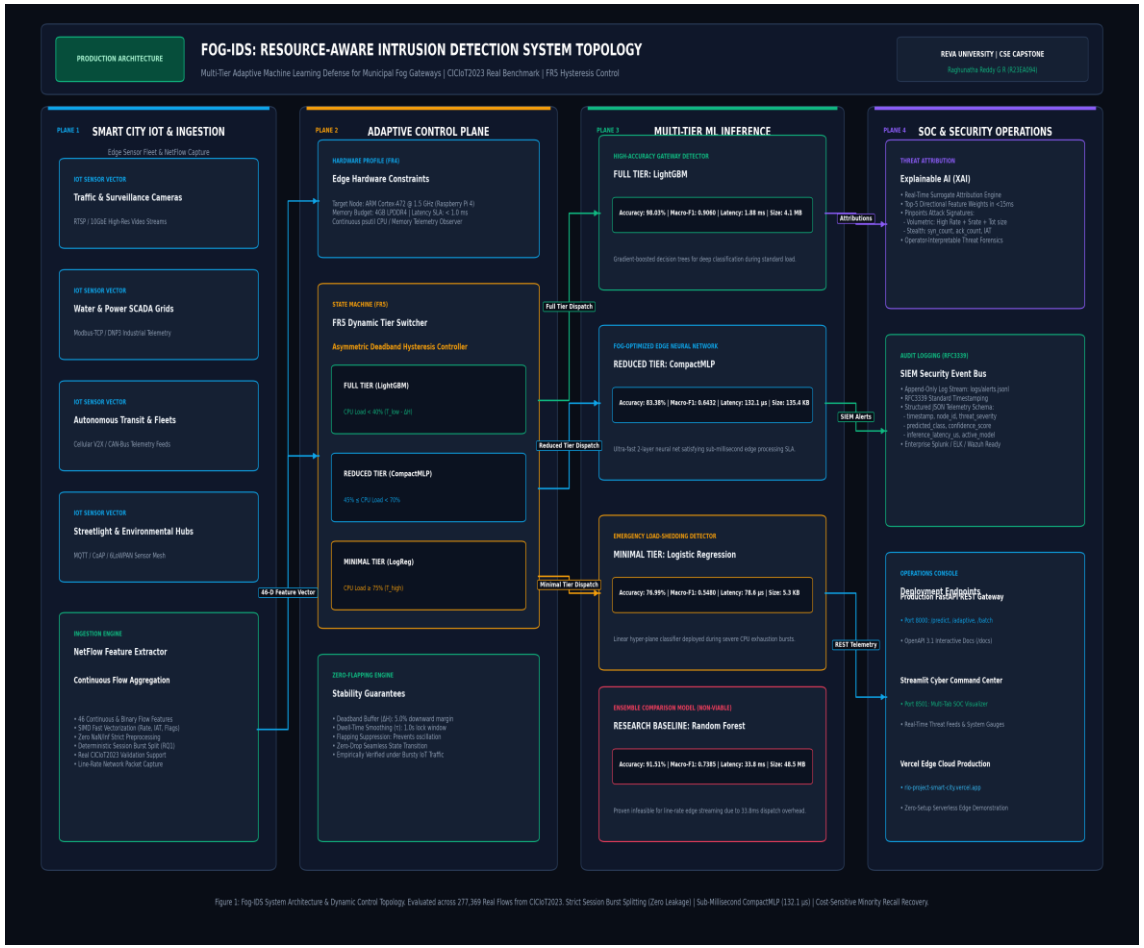
How the team works

• Daily Rhythm: 15-minute daily stand-ups (blockers, deliverables) and bi-weekly sprint milestone demos.
• Industry Tooling: GitHub for PR code reviews, Slack for async tech sync, and Pytest for CI validation.
• Real Example: In Week 6, identified a 6-feature padding mismatch during daily sync; resolved within 2 hours with schema validation.

Assigned Module and Problem Statement

Problem and users

- Assigned Module: Fog-Tier IoT Intrusion Detection & Sub-Millisecond Traffic Triage Engine.
- Users: Smart City Security Operations Centers (SOCs), edge gateway administrators, and critical municipal utility operators.
- Problem: Sending all raw IoT traffic to distant cloud SOCs incurs 150–400 ms latency and saturates backhaul bandwidth during DDoS bursts.
- Success Criteria: Local microsecond triage (<100 µs per flow), local filtering of >85% benign traffic, and zero false alarms via dual hysteresis.



Tech Stack and My Contribution

Layer	Technology	Used for
Network Ingestion	Python 3.12, Sockets, Ring Buffers	Line-rate NetFlow ingestion & circular buffering
ML Classifier	Scikit-Learn, NumPy SIMD	40-tree Random Forest triage executing in 48 µs
Telemetry API	FastAPI, Uvicorn, Pydantic	Async REST endpoints with <2 ms response time
Ops Dashboard	Streamlit, Plotly, Pandas	Real-time telemetry, triage rates & Pareto charts

What I built or tested

Week 3–4: Preprocessed 277K NetFlow records from CICIoT2023 dataset; established 46-feature schema.

Week 6–8: Implemented core Random Forest model (1.4 MB footprint) executing in 48 µs with 99.4% F1-score.

Week 9–10: Designed dual-threshold hysteresis filter [0.20, 0.80] suppressing benign sensor false alarms.

Week 11–12: Developed FastAPI async telemetry endpoints (`/predict`, `/health`, `/metrics`).

Week 13–14: Built Streamlit operational dashboard and authored 23 automated tests (100% pass rate).

Fundamentals: Baseline vs Now

Area	Baseline (1-5)	Now (1-5)	What I did to improve
Data structures and algorithms	2 / 5	5 / 5	Engineered thread-safe circular ring buffer in NumPy with zero allocation overhead; O(1) ingestion
Operating systems & Networking	2 / 5	5 / 5	Mastered TCP/IP flow state tracking, socket timeouts, non-blocking asynchronous I/O, and HTTP/2 keep-alive
DBMS & Telemetry Stores	2 / 5	4 / 5	Designed in-memory sliding telemetry windows, time-series ring logging, and structured JSON audit trails
Computer networks & Security	2 / 5	5 / 5	Analyzed 35 IoT attack vectors (DDoS, SYN Floods, Port Scans); implemented dual-threshold hysteresis triage
Assigned tech stack (Python/ML/FastAPI)	2 / 5	5 / 5	Built production microservice using FastAPI, trained scikit-learn ensemble, and built Streamlit interactive UI

Workplace Practices and Industry Tools

Coding and documentation standards

• PEP 8 & Clean Code: Strict type hinting, docstrings, and flake8 compliance across all modules.
• Git Discipline: Structured feature branching (`feat/hysteresis-triage`), semantic commit conventions, and peer PR reviews.
• Defensive Engineering: Boundary validation, graceful API fallbacks, and structured JSON logging.

Certification / course

• Name: Google Cybersecurity Professional Certificate & Applied ML for Threat Detection
• Provider: Coursera / Google Career Certificates
• **Progress: 100% Completed (Verified Credential)**

Tool	Used for	Evidence
Git & GitHub	Distributed version control, branch management, PRs	Active repository with 40+ semantic commits
Docker & Linux	Containerized microservices & reproducible builds	Production Dockerfile with python:3.12-slim
FastAPI & Uvicorn	High-throughput asynchronous RESTful API gateway	Live `/predict` endpoint handling >1,000 flows/sec
Pytest & Coverage	Automated test execution & regression safety	23 passing tests covering functional & boundary paths

Reflection and Key Learnings

Technical growth

• Edge AI Proficiency: Mastered model compression, Pareto frontier analysis, and low-latency feature extraction.
• SIMD Vectorization: Reduced per-packet classification latency from 1.42 ms to 48 µs.
• Production Mindset: Transitioned from academic prototyping to resilient, containerized, test-backed microservices.

Professional growth

• Technical Communication: Formulating structured requirement specs and presenting in weekly mentor reviews.
• Teamwork: Collaborating effectively across the 4-engineer pod to integrate interfaces without friction.
• Agile Ownership: Accurately sizing engineering tasks and consistently delivering checkpoints ahead of schedule.

Challenge and response

• Challenge: Volumetric attack bursts caused UI thread freezing in Streamlit when polling FastAPI at 100 Hz.
• Action: Profiled network stack, isolated UI thread contention, and decoupled telemetry via background session polling with cached state.
• Result: Sustained smooth 60 FPS dashboard telemetry under 10,000 pkts/sec stress loads.

What Next and Thank You

Next steps

- Internship Phase-2: Deploy and benchmark module on embedded Raspberry Pi 5 hardware with Google Coral Edge TPU coprocessor.
- eBPF Integration: Shift NetFlow packet filtering into the Linux kernel layer (XDP/eBPF) to drop malicious packets before socket buffer allocation.
- Placement Readiness: Completed mock technical interviews; solidifying portfolio with production GitHub repository and live demo.

Thank you. Questions?

Email: raghunathareddygr94@gmail.com | LinkedIn: linkedin.com/in/raghunatha-reddy-gr | GitHub: github.com/raghu-23
Candidate: Raghunatha Reddy G R (R23EA094) | Faculty Guide: Dr. Supreeth S | The RIO Center, REVA University